

Making Variant Greater Equal

Document Number: P0393r3, ISO/IEC JTC1 SC22 WG21

Audience: LWG

Date: 2016-06-21

Author: Tony Van Eerd (variant at forecode.com)

Motivation

These edits align `variant` with `optional` in making the relational operators delegate to the relational operators of the underlying types

drive by fixes:

Other fixes, noticed when editing and/or suggested by LWG during review

- `operator<()` was defined as if `index()` was signed. It is not.
- convert *Effects* to *Returns* if...otherwise...otherwise... (as it was getting complicated)

?6 Relational operators

[variant.relops]

```
template <class... Types> constexpr bool operator==(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get< i >(v) == get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `(v.valueless_by_exception() && w.valueless_by_exception()) || (v.index() == w.index() && get< i >(v) == get< i >(w))` with `i` being `v.index()`.~~

Returns: If `v.index()` != `w.index()`, false; otherwise if `v.valueless_by_exception()`, true; otherwise `get< i >(v) == get< i >(w)` with `i` being `v.index()`.

```
template <class... Types> constexpr bool operator!=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get< i >(v) != get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `!(v == w)`.~~

Returns: If `v.index()` != `w.index()`, true; otherwise if `v.valueless_by_exception()`, false; otherwise `get< i >(v) != get< i >(w)` with `i` being `v.index()`.

```
template <class... Types> constexpr bool operator<(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get< i >(v) < get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `(v.index() < w.index()) || (v.index() == w.index() && !v.valueless_by_exception() && get< i >(v) < get< i >(w))` with `i` being `v.index()`.~~

Returns: If `w.valueless_by_exception()`, false; otherwise if `v.valueless_by_exception()`, true; otherwise, if `v.index() < w.index()`, true; otherwise if `v.index() > w.index()`, false; otherwise `get< i >(v) < get< i >(w)` with `i` being `v.index()`.

```
template <class... Types> constexpr bool operator>(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get< i >(v) > get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `w < v`.~~

Returns: If `v.valueless_by_exception()`, false; otherwise if `w.valueless_by_exception()`, true; otherwise, if `v.index() > w.index()`, true; otherwise if `v.index() < w.index()`, false; otherwise `get< i >(v) > get< i >(w)` with `i` being `v.index()`.

```
template <class... Types> constexpr bool operator<=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get< i >(v) <= get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `!(v > w)`.~~

Returns: If `v.valueless_by_exception()`, true; otherwise if `w.valueless_by_exception()`, false; otherwise, if `v.index() < w.index()`, true; otherwise if `v.index() > w.index()`, false; otherwise `get< i >(v) <= get< i >(w)` with `i` being `v.index()`.

```
template <class... Types> constexpr bool operator>=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires: `get<i>(v) >= get<i>(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Effects: ~~Equivalent to return `!(v < w)`:~~

Returns: If `w.valueless_by_exception()`, `true`; otherwise if `v.valueless_by_exception()`, `false`; otherwise, if `v.index() > w.index()`, `true`; otherwise if `v.index() < w.index()`, `false`; otherwise `get<i>(v) >= get<i>(w)` with `i` being `v.index()`.